

```
...
>>> formMolecules(unobtainium=12)
'aether'
```

As you can see, the `def` statement ❶ is the same as before, and only the function's code ❷ needed updating. When you use the `**` syntax, the `def` statement and the function calls become much simpler to write and still produce readable code.

Using * and ** to Create Wrapper Functions

A common use case for the `*` and `**` syntax in `def` statements is to create wrapper functions, which pass on arguments to another function and return that function's return value. You can use the `*` and `**` syntax to forward any and all arguments to the wrapped function. For example, we can create a `printLowercase()` function that wraps the built-in `print()` function. It relies on `print()` to do the real work but converts the string arguments to lowercase first:

```
❶ >>> def printLower(*args, **kwargs):
❷ ...     args = list(args)
...     for i, value in enumerate(args):
...         args[i] = str(value).lower()
❸ ...     return print(*args, **kwargs)
...
>>> name = 'Albert'
>>> printLower('Hello,', name)
hello, albert
>>> printLower('DOG', 'CAT', 'MOOSE', sep=', ')
dog, cat, moose
```

The `printLower()` function ❶ uses the `*` syntax to accept a varying number of positional arguments in a tuple assigned to the `args` parameter, whereas the `**` syntax assigns any keyword arguments to a dictionary in the `kwargs` parameter. If a function uses `*args` and `**kwargs` together, the `*args` parameter must come before the `**kwargs` parameter. We pass these on to the wrapped `print()` function, but first our function modifies some of the arguments, so we create a list form of the `args` tuple ❷.